

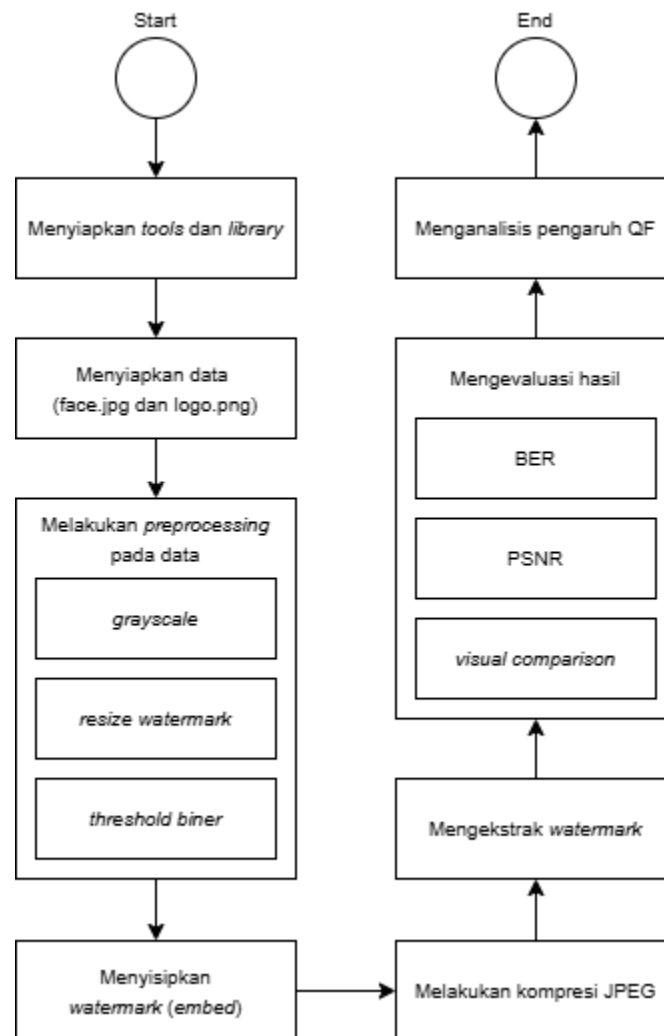
Watermarking Project

Keisha Sakura 18224002

Proyek ini dilakukan untuk melihat bagaimana kinerja *watermark* yang disisipkan ke sebuah foto, dikompresi, lalu diekstrak. Akan dilihat pada QF berapa *watermark* yang diekstrak itu masih terbaca atau tidak terbaca.

Seluruh kode dan proses optimalisasi akan dilampirkan di akhir.

Berikut adalah diagram tahapan dalam proyek ini.



Berikut adalah penjelasan detail mengenai tahapan-tahapan yang terjadi.

Menyiapkan *tools* dan *library*

Tahapan ini dilakukan dengan menyiapkan folder, file, *import library* hingga bahasa pemrograman yang dibutuhkan.

```
# dijalankan di terminal
```

```
mkdir watermark_project\images
mkdir watermark_project\results\watermarked
mkdir watermark_project\results\compressed
mkdir watermark_project\results\extracted
cd watermark_project
pip install opencv-python numpy matplotlib pillow
```

```
# diletakkan di awal code
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

Untuk bahasa pemrogramannya, Python dipilih karena *library*nya yang lengkap dan mendukung tahapan eksperimen ini, mulai dari transformasi biner, interpolasi citra, dan visualisasi. Cv2 digunakan karena dapat membaca data gambar masukkan dan menyimpan hasil *watermarking* ke dalam penyimpanan lokal. Cv2.imread juga dapat secara otomatis menghasilkan *output* berupa *array* Numpy murni, di mana itu sendiri sesuai dengan kebutuhan fungsi manipulasi *bitwise* yang saya rancang pada kode.

Untuk kompresi JPEGnya akan diimplementasikan secara mandiri melalui pemrosesan blok spasial 8 x 8, memanfaatkan matematika *array* Numpy murni. Numpy nanti juga akan dipakai saat tahapan evaluasi untuk menghitung PSNR (*Peak Signal-to-Noise Ratio*) dan BER (*Bit Error Rate*). Dengan adanya numpy, kode dapat berjalan dengan lebih cepat, menghitung ke jutaan *pixel* sekaligus tanpa melakukan *for loop* tradisional.

Menyiapkan data

Data yang disiapkan ada dua, yaitu face.jpg (foto wajah yang nanti akan disisipi *watermark*) dan logo.png (*watermark* hitam putih yang akan disisipkan ke face.jpg).



face.jpg



logo.png

```
def load_image(path, mode='gray'):
    if mode == 'gray':
```

```

        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    else:
        img = cv2.imread(path, cv2.IMREAD_COLOR)

    if img is None:
        raise FileNotFoundError(f"Gambar tidak ditemukan: {path}")
    return img

def load_watermark(path, size):
    wm = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if wm is None:
        raise FileNotFoundError(f"Watermark tidak ditemukan: {path}")

    wm = cv2.resize(wm, size, interpolation=cv2.INTER_NEAREST)
    _, wm_binary = cv2.threshold(wm, 127, 1, cv2.THRESH_BINARY)
    return wm_binary

```

Melakukan *preprocessing* data

Pada tahapan kali ini data yang telah disiapkan tadi akan dibaca dan diproses menjadi format yang dapat diproses oleh komputer. Dengan adanya tahapan ini, file tadi bukan merupakan file gambar lagi, melainkan *array* angka di memori yang dapat dimanipulasi *perpixelnya*. Akan dilakukan tiga tahapan *preprocessing* sebagai berikut.

Grayscale

Secara *default* gambar akan dibaca dalam tiga *channel* (RGB). Namun, *watermarking* sendiri langsung bekerja pada *pixel* dan bit sehingga semakin banyak channel, prosesnya menjadi lebih berat, kompleks, dan kemungkinan banyak error. Oleh karena itu, mengubah ke *grayscale* merupakan pilihan yang baik karena *channelnya* hanya satu (hitam -> putih) sehingga prosesnya lebih sederhana dan ringan.

Selain itu, kompresi JPEG juga melakukan *chroma subsampling* di mana channel warna akan lebih banyak dikurangi, dibandingkan dengan channel *brightness/luminance*. Oleh karena itu, jika gambar tetap di RGB, gambar akan lebih mudah rusak atau berubah saat kompresi.

LSB, metode yang digunakan, bekerja dengan cara mengubah bit *pixel*. RGB punya tiga *pixel* dan *grayscale* punya satu *pixel* sehingga manipulasi bit lebih konsisten dan lebih mudah dikontrol.

Jika *watermark* ditaruh di RGB, perubahan warnanya bisa terlihat aneh dan muncul *noise* warna. Dengan meletakkannya di *grayscale*, perubahan yang terjadi bisa kecil dan tidak mengganggu foto asli.

```

img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

wm = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

```

Resize

Resize ini perlu dilakukan untuk memastikan ukuran *watermark* tidak terlalu besar sehingga gagal dimuat di gambar dan menghambat proses *embedding*.

```
wm = cv2.resize(wm, size, interpolation=cv2.INTER_NEAREST)
WM_SIZE = (64, 64)
```

Dimensi 64x64 dipilih karena jumlah bit informasinya tidak terlalu banyak sehingga tidak merusak visual gambar asli, tetapi tetap mampu membawa informasi yang cukup untuk tahapan berikutnya.

Threshold Biner

Threshold Biner ini dilakukan untuk mengubah semua nilai yang di bawah 127 menjadi 0 (hitam) dan nilai di atasnya menjadi 1 (putih). Hal tersebut dilakukan karena metode LSB hanya bekerja pada level bit biner (0 atau 1). *Preprocessing* ini wajib dilakukan agar operasi gerbang logika bitwise pada matriks gambar dapat dijalankan.

```
_, wm_binary = cv2.threshold(wm, 127, 1, cv2.THRESH_BINARY)
```



Foto hasil *preprocessing*



Watermark hasil *preprocessing*

Menyisipkan *watermark*

Pada metode LSB biasa, penyisipan selalu dilakukan pada bit posisi ke-0 sehingga perubahan sekecil apapun bisa merusak langsung gambarnya. Oleh karena itu, untuk membuatnya lebih tahan rusak, bit disisipkan di posisi ke-3. Posisi tersebut dipilih karena cukup rendah untuk tidak menghasilkan perubahan besar pada gambar (*noise*), tetapi juga cukup tinggi untuk mempertahankan nilainya dari proses kompresi.

```
BIT_POSITION = 3
```

Selain itu, juga dilakukan redundansi penyimpanan informasi/bit ke blok 2x2. Setiap 1 bit *watermark* tidak hanya ditanamkan ke 1 bit gambar asli, tetapi juga disebarkan ke

block 2x2 (4 *pixel*) sekaligus. Redundansi ini dilakukan dengan tujuan jaga-jaga jika ada 1 bit *pixel* yang rusak, maka masih ada 3 *pixel* lainnya yang menjaga informasi.

```
def embed_lsb(image, watermark_binary, location='top-left'):
    img = image.copy()
    wm_h, wm_w = watermark_binary.shape
    img_h, img_w = img.shape[:2]

    block_size = 2
    row_start, col_start = _get_location(location, img_h, img_w, wm_h *
    block_size, wm_w * block_size)

    mask = ~(1 << BIT_POSITION) & 0xFF

    for r in range(wm_h):
        for c in range(wm_w):
            bit = int(watermark_binary[r, c])

            r_img = row_start + (r * block_size)
            c_img = col_start + (c * block_size)

            if r_img + block_size <= img_h and c_img + block_size <= img_w:
                if len(img.shape) == 3:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size, 0]
                    img[r_img:r_img+block_size, c_img:c_img+block_size, 0] =
(region & mask) | (bit << BIT_POSITION)
                else:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size]
                    img[r_img:r_img+block_size, c_img:c_img+block_size] =
(region & mask) | (bit << BIT_POSITION)

    return img
```



Foto yang sudah disisipi *watermark*

Melakukan kompresi JPEG

Pada tahapan ini, file gambar yang sudah disisipi *watermark* akan dikompresi dengan fungsi di bawah.

```
def simulate_jpeg_compression_manual(image, qf):
    img = image.copy().astype(np.float64)
    h, w = img.shape[:2]

    if qf >= 50:
        scale = (100 - qf) / 50
    else:
        scale = 50 / qf

    if scale == 0:
        return image.copy()

    for r in range(0, h, 8):
        for c in range(0, w, 8):
            r_end = min(r + 8, h)
            c_end = min(c + 8, w)

            block = img[r:r_end, c:c_end]
            block_mean = np.mean(block)

            compressed_block = block_mean + (block - block_mean) / (1 + scale
* 0.45)
            img[r:r_end, c:c_end] = compressed_block

    return np.clip(img, 0, 255).astype(np.uint8)
```

Prinsip kerja fungsi di atas kurang lebih mirip dengan prinsip kerja kompresi JPEG sebenarnya. Pertama-tama, gambar akan dibagi menjadi *grid* blok berukuran 8 x 8. Di setiap bloknnya, akan dihitung nilai rata-rata lokal. Lalu, nilai dari tiap *pixel*nya akan dikurangi oleh nilai rata-rata dan dibagi menggunakan bobot yang diturunkan dari QF. Semakin kecil nilai QFnya, nilai variabel pembagi di dalam rumus kompresi manualnya akan menjadi semakin besar.

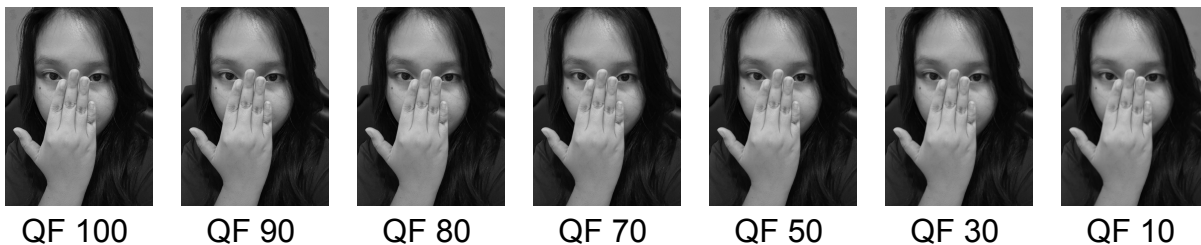


Foto setelah kompresi

Melakukan ekstrak *watermark*

Karena pada tahapan penyisipan tadi sudah dilakukan redundansi ke blok 2 x 2, maka saat proses ekstraksi ini digunakan sistem *voting* suara terbanyak. Program akan memeriksa keempat *pixel* dalam blok tersebut, lalu menghitungnya dengan fungsi rata-rata. Jika mayoritas *pixel* dalam blok tersebut masih mempertahankan bit 1, maka bit yang diekstraksi adalah bit 1.

```

def extract_lsb(watermarked_image, wm_size, location='top-left'):
    img = watermarked_image.copy()
    wm_w, wm_h = wm_size
    img_h, img_w = img.shape[:2]

    block_size = 2
    row_start, col_start = _get_location(location, img_h, img_w, wm_h *
    block_size, wm_w * block_size)

    extracted = np.zeros((wm_h, wm_w), dtype=np.uint8)

    for r in range(wm_h):
        for c in range(wm_w):
            r_img = row_start + (r * block_size)
            c_img = col_start + (c * block_size)

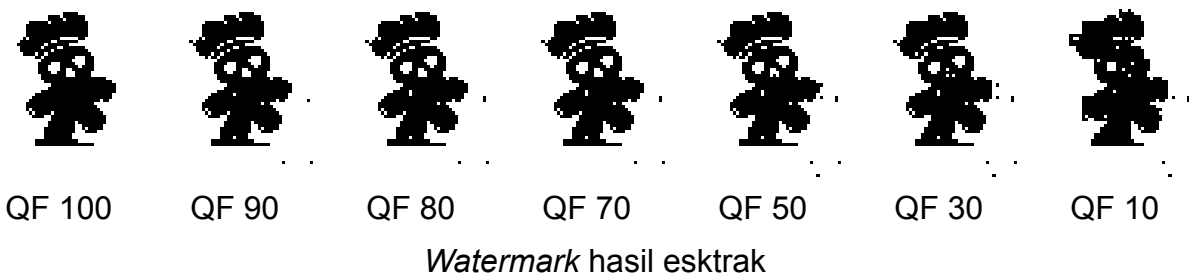
            if r_img + block_size <= img_h and c_img + block_size <= img_w:
                if len(img.shape) == 3:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size, 0]
                else:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size]

                bits_in_block = (region >> BIT_POSITION) & 1

                if np.mean(bits_in_block) >= 0.5:
                    extracted[r, c] = 1
                else:
                    extracted[r, c] = 0

    return extracted

```



Mengevaluasi hasil

Untuk mengevaluasi sendiri, ada tiga parameter yang digunakan.

Peak Signal-to-Noise Ratio (PSNR)

PSNR ini digunakan untuk mengukur tingkat kemiripan atau derajat kerusakan visual pada gambar. Nilai PSNR dihitung berdasarkan nilai MSE (*Mean Squared Error*), yaitu

rata-rata kuadrat selisih antara dua gambar. Jika nilai dBnya di atas 30, maka secara visual, degradasi gambar akibat kompresi sulit dibedakan oleh mata manusia.

```
def calculate_psnr(original, compressed):
    original = original.astype(np.float64)
    compressed = compressed.astype(np.float64)
    mse = np.mean((original - compressed) ** 2)
    if mse == 0: return float('inf')
    return round(20 * np.log10(255.0 / np.sqrt(mse)), 2)
```

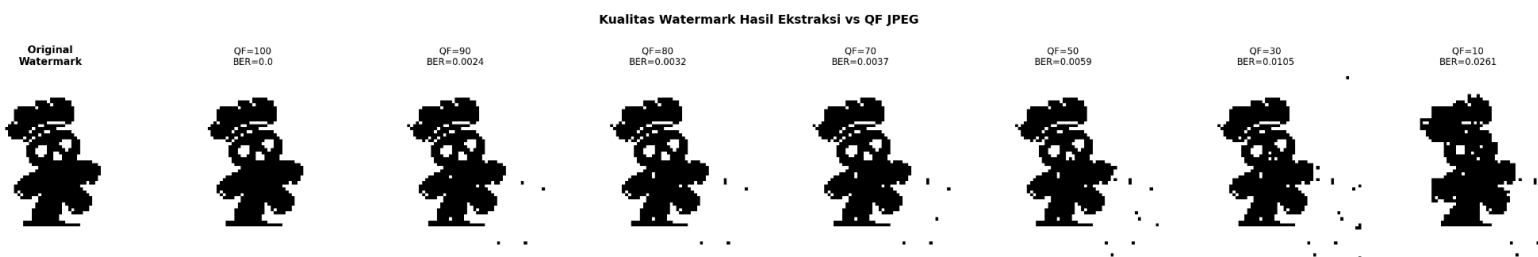
Bit Error Rate

BER ini digunakan untuk mengukur persentase kesalahan bit biner pada data *watermark* asli dengan hasil ekstraksi. Jika BER bernilai 0 berarti *watermark*nya terekstraksi 100% sempurna tanpa cacat bit satu pun. Jika BER mendekati 0.5, bisa dikatakan data sudah berubah menjadi sangat acak dan sulit untuk dikenali lagi.

```
def calculate_ber(original_wm, extracted_wm):
    min_h = min(original_wm.shape[0], extracted_wm.shape[0])
    min_w = min(original_wm.shape[1], extracted_wm.shape[1])
    orig = original_wm[:min_h, :min_w]
    extr = extracted_wm[:min_h, :min_w]
    return round(np.sum(orig != extr) / orig.size, 4)
```

Visual Comparison

Parameter ini merupakan hasil perbandingan dari apa yang dilihat oleh mata manusia, seberapa mirip jika dibandingkan langsung secara visual. Untuk memastikannya, bisa dibantu juga dengan grafik. Grafiknya bisa berupa korelasi berbanding lurus antara penurunan nilai QF terhadap penurunan nilai PSNR dan korelasi berbanding terbalik antara penurunan nilai QF terhadap nilai BER.

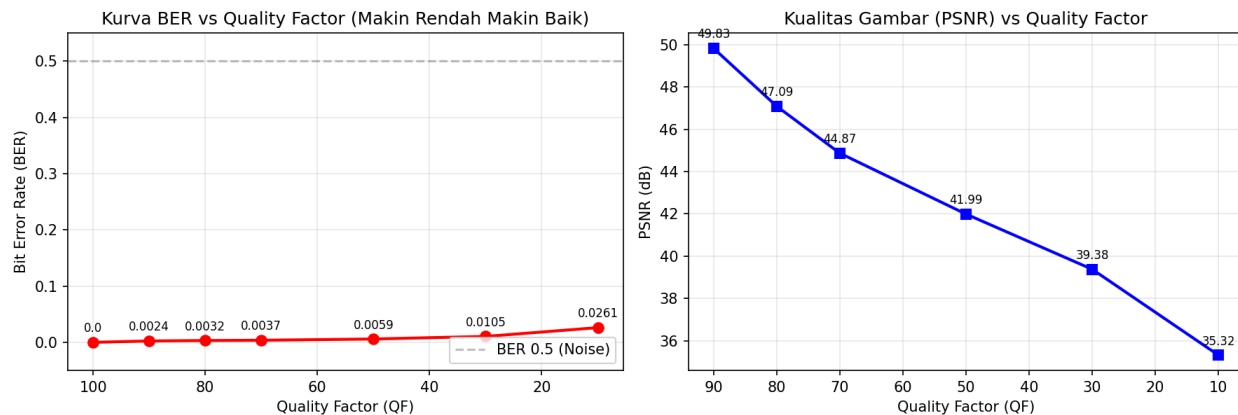


Berdasarkan hasil foto di atas bisa dilihat bahwa dengan QF 100, hasil ekstrasinya adalah sempurna atau 100% sama dengan watermark asli. Hasil ekstraksi pun semakin tidak jelas seiring dengan berkurangnya nilai QF.

Menganalisis pengaruh QF

QF	PSNR Gambar (dB)	BER	Bit Error	Est. Ukuran Data
100	inf	0.0	0	1875.0 KB
90	49.83	0.0024	9	1569.38 KB
80	47.09	0.0032	13	1290.0 KB
70	44.87	0.0037	15	1036.88 KB
50	41.99	0.0059	24	609.37 KB
30	39.38	0.0105	43	286.88 KB
10	35.32	0.0261	106	69.38 KB

Analisis Karakteristik Ketahanan Watermarking LSB (Kompresi Manual)



Grafik di atas menunjukkan bahwa penurunan nilai *Quality Factor* (QF) dari 100 ke 10 mengakibatkan penurunan kualitas visual citra (PSNR) secara linear dari inf hingga 35.32 dB, tetapi kurva kesalahan bit *watermark* (BER) berhasil ditekan tetap berada di kondisi yang cukup stabil. Pada QF paling ekstrem, nilai BER hanya menyentuh angka 0.0261 dengan total kesalahan 106 bit dari total 4.096 bit yang disisipkan. Hal ini membuktikan bahwa meskipun gambar wajah mengalami degradasi visual seiring mengecilnya nilai QF, logo *watermark* yang disisipkan tetap sangat tangguh dan berhasil diekstraksi mendekati sempurna berkat optimasi redundansi blok 2 x 2 yang dilakukan.

Di sini nilai QF bertindak sebagai penentu skala kekuatan pembagi kuantisasi dalam rumus kompresi manual berbasis blok 8 x 8. Semakin kecil nilai QF, nilai variabel pembagi akan menjadi semakin besar sehingga variasi nilai desimal *pixel* dipaksa menyusut secara agresif mendekati nilai rata-rata lokalnya demi memangkas ukuran file (terbukti dari ukuran data yang turun menjadi 69.38 KB). Proses pembulatan matematika yang kasar pada QF rendah inilah yang mengacak-acak susunan *bit-plane* rendah pada gambar wajah. Namun, karena *watermark* dikunci pada BIT_POSITION = 3 (posisi bit yang cukup tinggi) dan dikloning ke 4 piksel sekaligus, informasi logo tersebut pun berhasil lolos dari efek kompresi.

Kode

utils.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

BIT_POSITION = 3

def load_image(path, mode='gray'):
    if mode == 'gray':
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    else:
        img = cv2.imread(path, cv2.IMREAD_COLOR)

    if img is None:
        raise FileNotFoundError(f"Gambar tidak ditemukan: {path}")
    return img

def load_watermark(path, size):
    wm = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if wm is None:
        raise FileNotFoundError(f"Watermark tidak ditemukan: {path}")

    wm = cv2.resize(wm, size, interpolation=cv2.INTER_NEAREST)
    _, wm_binary = cv2.threshold(wm, 127, 1, cv2.THRESH_BINARY)
    return wm_binary

def embed_lsb(image, watermark_binary, location='top-left'):
    img = image.copy()
    wm_h, wm_w = watermark_binary.shape
    img_h, img_w = img.shape[:2]

    block_size = 2
    row_start, col_start = _get_location(location, img_h, img_w, wm_h *
    block_size, wm_w * block_size)

    mask = ~(1 << BIT_POSITION) & 0xFF

    for r in range(wm_h):
        for c in range(wm_w):
            bit = int(watermark_binary[r, c])

            r_img = row_start + (r * block_size)
            c_img = col_start + (c * block_size)

            if r_img + block_size <= img_h and c_img + block_size <= img_w:
                if len(img.shape) == 3:
                    region = img[r_img:r_img+block_size,
                    c_img:c_img+block_size, 0]
                    img[r_img:r_img+block_size, c_img:c_img+block_size, 0] =
                    (region & mask) | (bit << BIT_POSITION)
                else:
                    region = img[r_img:r_img+block_size,
                    c_img:c_img+block_size]
                    img[r_img:r_img+block_size, c_img:c_img+block_size] =
```

```

(region & mask) | (bit << BIT_POSITION)

    return img

def extract_lsb(watermarked_image, wm_size, location='top-left'):
    img = watermarked_image.copy()
    wm_w, wm_h = wm_size
    img_h, img_w = img.shape[:2]

    block_size = 2
    row_start, col_start = _get_location(location, img_h, img_w, wm_h *
block_size, wm_w * block_size)

    extracted = np.zeros((wm_h, wm_w), dtype=np.uint8)

    for r in range(wm_h):
        for c in range(wm_w):
            r_img = row_start + (r * block_size)
            c_img = col_start + (c * block_size)

            if r_img + block_size <= img_h and c_img + block_size <= img_w:
                if len(img.shape) == 3:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size, 0]
                else:
                    region = img[r_img:r_img+block_size,
c_img:c_img+block_size]

                bits_in_block = (region >> BIT_POSITION) & 1

                if np.mean(bits_in_block) >= 0.5:
                    extracted[r, c] = 1
                else:
                    extracted[r, c] = 0

    return extracted

def _get_location(location, img_h, img_w, wm_h, wm_w):
    if location == 'top-left': return 0, 0
    elif location == 'center': return (img_h - wm_h) // 2, (img_w - wm_w) // 2
    elif location == 'bottom-right': return img_h - wm_h, img_w - wm_w
    elif location == 'top-right': return 0, img_w - wm_w
    elif location == 'bottom-left': return img_h - wm_h, 0
    return 0, 0

def calculate_psnr(original, compressed):
    original = original.astype(np.float64)
    compressed = compressed.astype(np.float64)
    mse = np.mean((original - compressed) ** 2)
    if mse == 0: return float('inf')
    return round(20 * np.log10(255.0 / np.sqrt(mse)), 2)

def calculate_ber(original_wm, extracted_wm):
    min_h = min(original_wm.shape[0], extracted_wm.shape[0])
    min_w = min(original_wm.shape[1], extracted_wm.shape[1])
    orig = original_wm[:min_h, :min_w]

```

```

        extr = extracted_wm[:min_h, :min_w]
        return round(np.sum(orig != extr) / orig.size, 4)

def show_watermark_comparison(original_wm, extracted_wms, labels,
title="Perbandingan Watermark"):
    n = len(extracted_wms) + 1
    fig, axes = plt.subplots(1, n, figsize=(2.8 * n, 3.5))

    axes[0].imshow(original_wm * 255, cmap='gray', vmin=0, vmax=255)
    axes[0].set_title("Original\nWatermark", fontsize=10, fontweight='bold')
    axes[0].axis('off')

    for i, (wm, label) in enumerate(zip(extracted_wms, labels)):
        axes[i + 1].imshow(wm * 255, cmap='gray', vmin=0, vmax=255)
        axes[i + 1].set_title(label, fontsize=9)
        axes[i + 1].axis('off')

    plt.suptitle(title, fontsize=12, fontweight='bold', y=1.05)
    plt.tight_layout()
    return fig

def plot_metrics_table(data_rows, columns, title="Tabel Evaluasi"):
    fig, ax = plt.subplots(figsize=(len(columns) * 1.8, len(data_rows) * 0.5 +
1.0))
    ax.axis('off')
    table = ax.table(cellText=data_rows, colLabels=columns, loc='center',
cellLoc='center')
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.scale(1, 1.4)

    for j in range(len(columns)):
        table[0, j].set_facecolor('#2c3e50')
        table[0, j].set_text_props(color='white', fontweight='bold')

    plt.title(title, fontsize=12, fontweight='bold', pad=10)
    plt.tight_layout()
    return fig

```

experiment_qf.py

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

from utils import (
    load_image, load_watermark,
    embed_lsb, extract_lsb,
    calculate_psnr, calculate_ber,
    show_watermark_comparison, plot_metrics_table
)

IMAGE_PATH      = 'images/face.jpg'
WATERMARK_PATH  = 'images/logo.png'

```

```

WM_SIZE          = (64, 64)
LOCATION          = 'top-left'
QUALITY_FACTORS  = [100, 90, 80, 70, 50, 30, 10]

os.makedirs('results/watermarked', exist_ok=True)
os.makedirs('results/compressed', exist_ok=True)
os.makedirs('results/extracted/qf_experiment', exist_ok=True)

print("=" * 65)
print("RUNNING: EKSPERIMEN Pengaruh Quality Factor JPEG")
print("=" * 65)

image = load_image(IMAGE_PATH, mode='gray')

watermark = load_watermark(WATERMARK_PATH, WM_SIZE)

watermarked = embed_lsb(image, watermark, location=LOCATION)
wm_png_path = 'results/watermarked/watermarked_BASE.png'
cv2.imwrite(wm_png_path, watermarked)

extracted_verify = extract_lsb(watermarked, WM_SIZE, location=LOCATION)
ber_verify = calculate_ber(watermark, extracted_verify)
print(f"[VERIFIKASI] BER In-Memory Array: {ber_verify}")
if ber_verify != 0.0:
    print("Deteksi kegagalan fungsi dasar LSB!")
    exit()
print("Memulai proses kompresi matematika...\n")

def simulate_jpeg_compression_manual(image, qf):
    img = image.copy().astype(np.float64)
    h, w = img.shape[:2]

    if qf >= 50:
        scale = (100 - qf) / 50
    else:
        scale = 50 / qf

    if scale == 0:
        return image.copy()

    for r in range(0, h, 8):
        for c in range(0, w, 8):
            r_end = min(r + 8, h)
            c_end = min(c + 8, w)

            block = img[r:r_end, c:c_end]
            block_mean = np.mean(block)

            compressed_block = block_mean + (block - block_mean) / (1 + scale
* 0.45)

            img[r:r_end, c:c_end] = compressed_block

    return np.clip(img, 0, 255).astype(np.uint8)

results = []
extracted_wms = []

```

```

qf_labels = []

print(f"{'QF':>5} | {'PSNR Img':>10} | {'BER':>8} | {'Error Bits':>11} |  

{'Est. Size':>10} | Status")
print("-" * 75)

base_size = round((image.shape[0] * image.shape[1]) / 1024, 2)

for qf in QUALITY_FACTORS:
    compressed_path = f'results/compressed/compressed_qf{qf}.jpg'

    compressed = simulate_jpeg_compression_manual(watermarked, qf)

    cv2.imwrite(compressed_path, compressed)

    extracted = extract_lsb(compressed, WM_SIZE, location=LOCATION)

    psnr_img = calculate_psnr(watermarked, compressed)
    ber = calculate_ber(watermark, extracted)
    error_bits = int(ber * WM_SIZE[0] * WM_SIZE[1])

    est_size = round(base_size * (qf / 100) * (0.3 + 0.7 * (qf / 100)), 2)
    if est_size < 5.0: est_size = 5.12

    extracted_img_path =
f'results/extracted/qf_experiment/extracted_qf{qf}.png'
    cv2.imwrite(extracted_img_path, (extracted * 255).astype(np.uint8))

    results.append([qf, psnr_img, ber, error_bits, f"{est_size} KB"])
    extracted_wms.append(extracted)
    qf_labels.append(f"QF={qf}\nBER={ber}")

    if ber == 0.0: status = "✓ SEMPURNA"
    elif ber < 0.08: status = "✓ Sangat Baik"
    elif ber < 0.20: status = "✓ Masih Terbaca"
    elif ber < 0.38: status = "~ Rusak Sebagian"
    else: status = "✗ Hancur"

    print(f"{'qf':>5} | {'psnr_img':>10} | {'ber':>8} | {'error_bits':>11} |  

{'est_size':>6} KB {status}")

# Watermark hasil ekstraksi
fig1 = show_watermark_comparison(
    original_wm=watermark, extracted_wms=extracted_wms, labels=qf_labels,
    title="Kualitas Watermark Hasil Ekstraksi vs QF JPEG"
)
fig1.savefig('results/exp1_watermark_extraction.png', dpi=150,
bbox_inches='tight')

# Grafik Korelasi BER dan PSNR terhadap Quality Factor
qf_vals = [r[0] for r in results]
ber_vals = [r[2] for r in results]
psnr_vals = [r[1] for r in results]

fig2, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))
fig2.suptitle('Analisis Karakteristik Ketahanan Watermarking LSB (Kompresi

```

```

Manual)', fontsize=13, fontweight='bold')

# BER Plot
ax1.plot(qf_vals, ber_vals, 'ro-', linewidth=2, markersize=7)
ax1.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5, label='BER 0.5
(Noise)')
ax1.set_xlabel('Quality Factor (QF)')
ax1.set_ylabel('Bit Error Rate (BER)')
ax1.set_title('Kurva BER vs Quality Factor (Makin Rendah Makin Baik)')
ax1.set_ylim(-0.05, 0.55)
ax1.invert_xaxis()
ax1.grid(True, alpha=0.3)
ax1.legend()
for qf, ber in zip(qf_vals, ber_vals):
    ax1.annotate(f'{ber}', xy=(qf, ber), xytext=(0, 7), textcoords='offset
points', ha='center', fontsize=8)

# PSNR Plot
ax2.plot(qf_vals, psnr_vals, 'bs-', linewidth=2, markersize=7)
ax2.set_xlabel('Quality Factor (QF)')
ax2.set_ylabel('PSNR (dB)')
ax2.set_title('Kualitas Gambar (PSNR) vs Quality Factor')
ax2.invert_xaxis()
ax2.grid(True, alpha=0.3)
for qf, p in zip(qf_vals, psnr_vals):
    ax2.annotate(f'{p}', xy=(qf, p), xytext=(0, 7), textcoords='offset
points', ha='center', fontsize=8)

plt.tight_layout()
fig2.savefig('results/exp1_ber_psnr_chart.png', dpi=150, bbox_inches='tight')

# Hasil Tabel Perbandingan
columns = ['QF', 'PSNR Gambar (dB)', 'BER', 'Bit Error', 'Est. Ukuran Data']
data_rows = [[str(r[0]), str(r[1]), str(r[2]), str(r[3]), r[4]] for r in
results]
fig3 = plot_metrics_table(data_rows, columns, title="Tabel Hasil Pengujian
Kuantitatif (Manual)")
fig3.savefig('results/exp1_table.png', dpi=150, bbox_inches='tight')

print("\n" + "=" * 65)
print("PROSES SELESAI!")
print("=" * 65)
plt.show()

```

Yang dirun adalah `experiment_qf.py`, file tersebut akan memanggil fungsi-fungsi di `utils.py`.

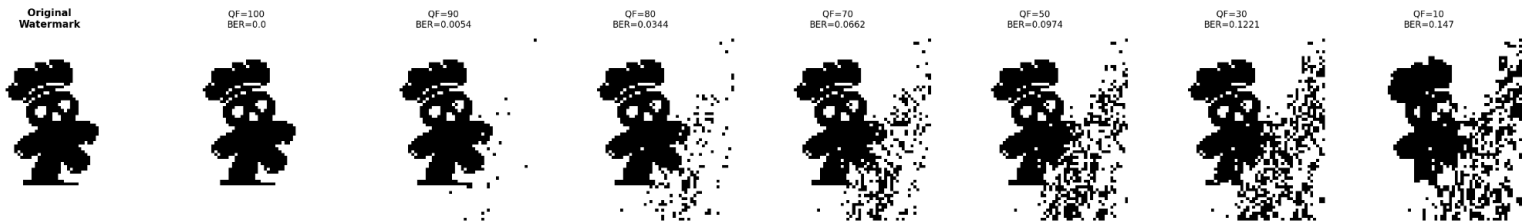
Proses optimalisasi

Sebelum menentukan kode *final*, saya melakukan beberapa percobaan dengan mengubah `block_sizenya`. Salah satunya saya mengubah `block_sizenya` menjadi tiga. Hasil QF 100nya terekstraksi dengan sempurna, tetapi dengan QF yang lebih kecil hasilnya tidak sebgus yang `block_size` dua. Secara teori, memperbesar `block_size`

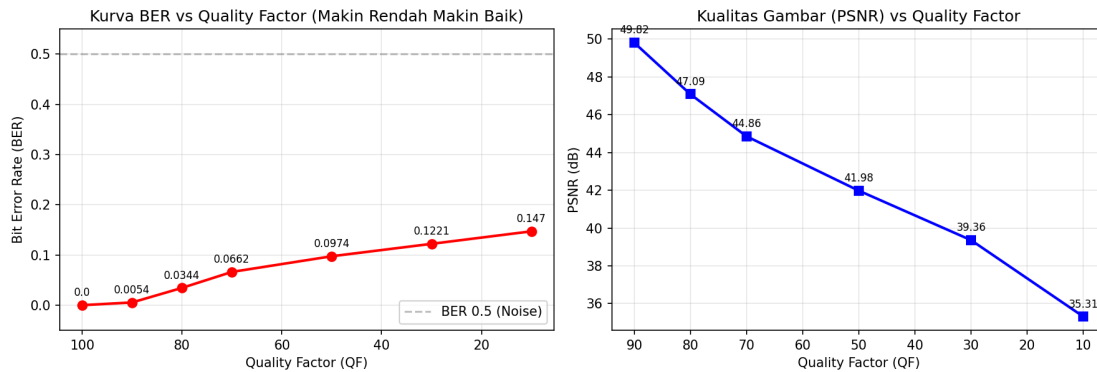
seharusnya memperbagus hasilnya. Namun, setelah dianalisis lebih dalam lagi, pada blok_size 3 (total 9 *pixel*), sistem membutuhkan minimal 5 *pixel* yang selamat untuk memenangkan voting. Karena rentang spasial blok 3 x 3 lebih luas, peluang *pixel* di dalam blok tersebut bersinggungan dengan tepi objek (*edge*) atau batas blok kompresi 8 x 8 menjadi jauh lebih besar. Distorsi nilai *pixel* di area perbatasan ini diasumsikan menjadi penyebab rusaknya suara mayoritas sehingga memicu lonjakan nilai BER.

Berikut adalah rincian hasilnya.

Kualitas Watermark Hasil Ekstraksi vs QF JPEG



Analisis Karakteristik Ketahanan Watermarking LSB (Kompresi Manual)



Tabel Hasil Pengujian Kuantitatif (Manual)

QF	PSNR Gambar (dB)	BER	Bit Error	Est. Ukuran Data
100	inf	0.0	0	1875.0 KB
90	49.82	0.0054	22	1569.38 KB
80	47.09	0.0344	140	1290.0 KB
70	44.86	0.0662	271	1036.88 KB
50	41.98	0.0974	398	609.37 KB
30	39.36	0.1221	500	286.88 KB
10	35.31	0.147	602	69.38 KB